

Machine Learning for Biology and Health

CSCI 1851
Spring 2026

Ritambhara Singh

March 10, 2026
Tuesday



Today's goal – Learn about classification using fully connected neural networks and some practical deep learning considerations

(1) **New task**: Predicting from medical images

(2) FCNN for classification

(3) Backpropagation in action

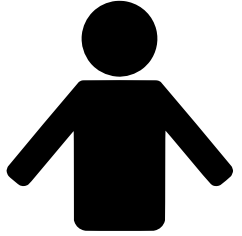
(4) Neural Networks as matrix operations

(5) Batching and broadcasting

(6) **Class activity**: Coding Demo in Pytorch

Predicting from medical images

Chest X-ray classification



Input: $\mathbb{X}$



How can we numerically represent images?



Function: f



$f(\mathbb{X}) \rightarrow \mathbb{Y}$

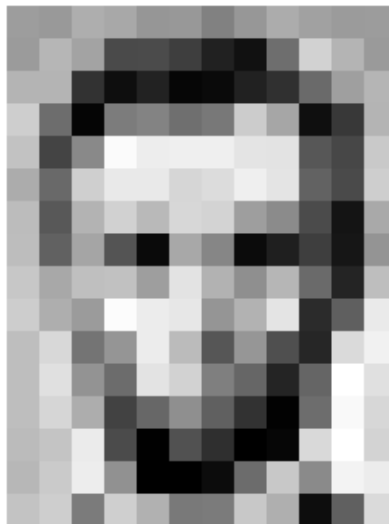
Target: $\mathbb{Y}$

Has pneumonia/
Does not have
pneumonia

Representing digits in the computer

- Numbers known as ***pixel values*** (a grid of discrete values that make up an image)

0 is white, 255 is black, and numbers in between are shades of gray



157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	93	17	110	210	180	154
180	180	50	14	94	6	10	93	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	85	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	95	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	85	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	95	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Our problem:

$Z = [0, 255]$
norm
R

Input: $\mathbb{X}$

Pixel Grid

$x^{(1)} =$



128x128 pixels

$x^{(2)} =$



What is our input space?

What is our output space?

What is our prediction task?

→ Function: f →

$f(\mathbb{X}) \rightarrow \mathbb{Y}$

yes {0, 1}

Target: $\mathbb{Y}$

Has Pneumonia?

$y^{(1)} = 0$



$y^{(2)} = 1$



What is our input??

$5 \times 5 = 25$

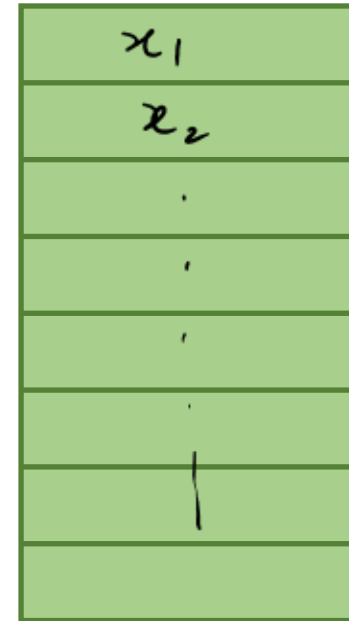
Flatten

1D ~~array~~ array

2D



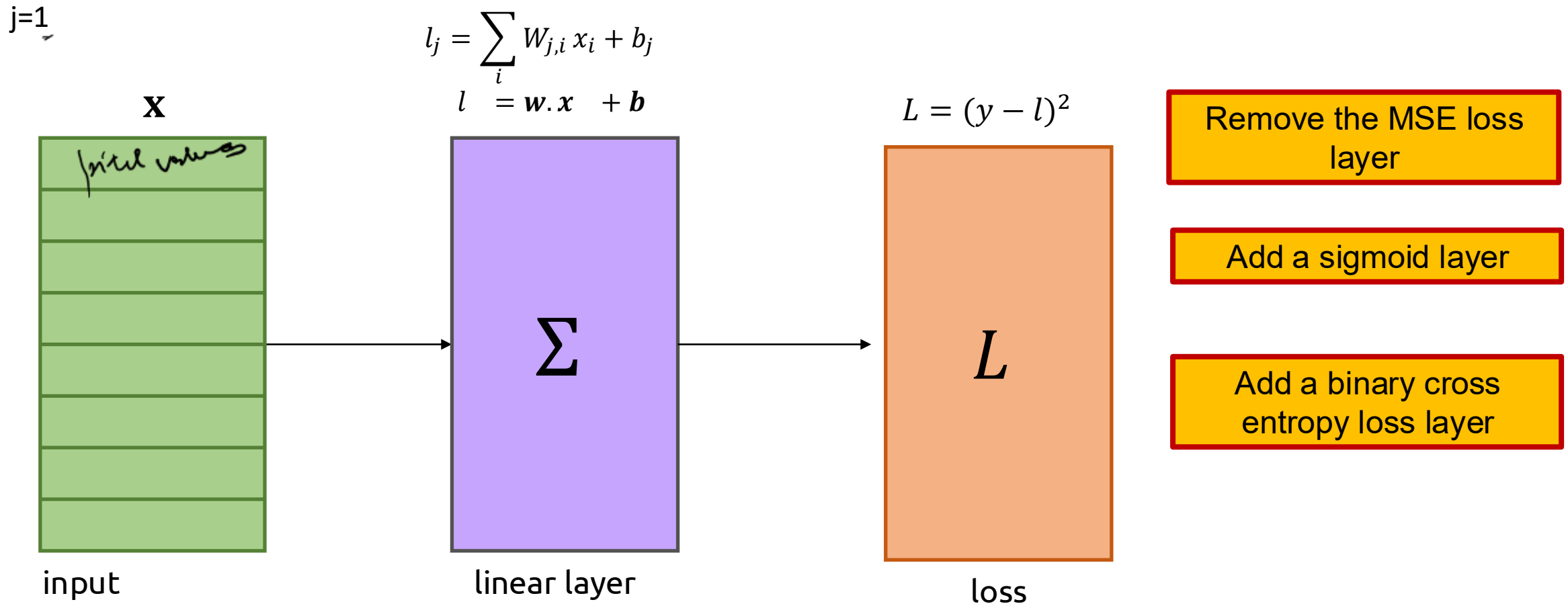
157	53	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	106	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218



input

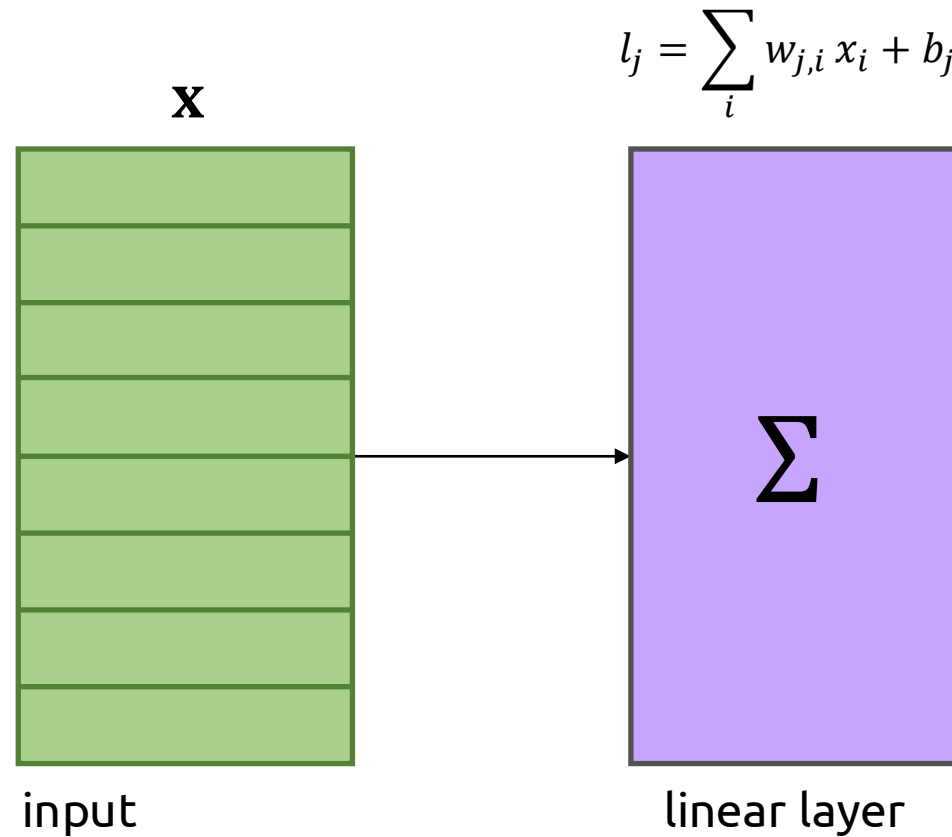
Building our FCNN for classification

What changes do we make for this task?



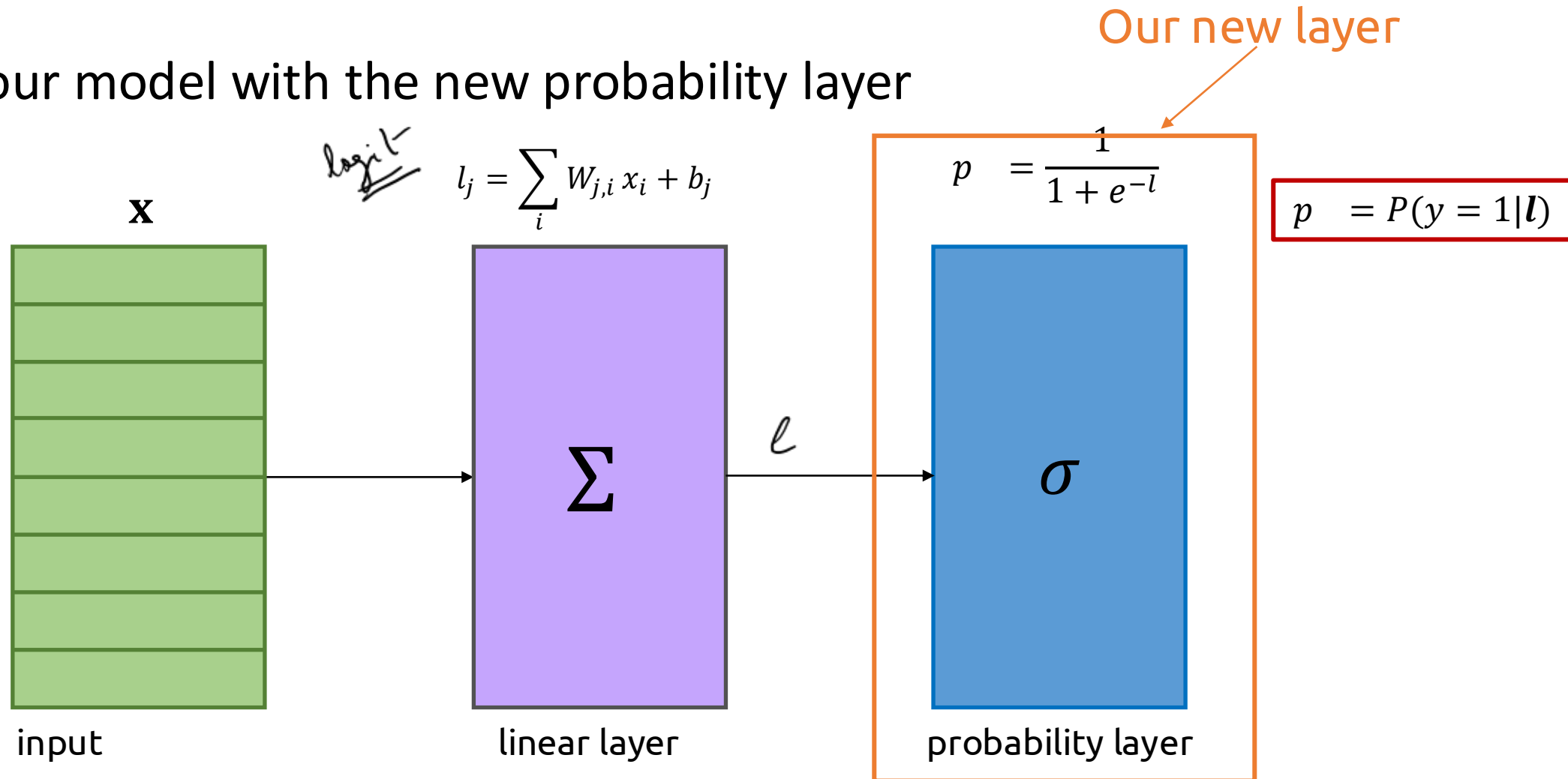
Our model before

- This is a simplified view of our model with an input and a linear layer



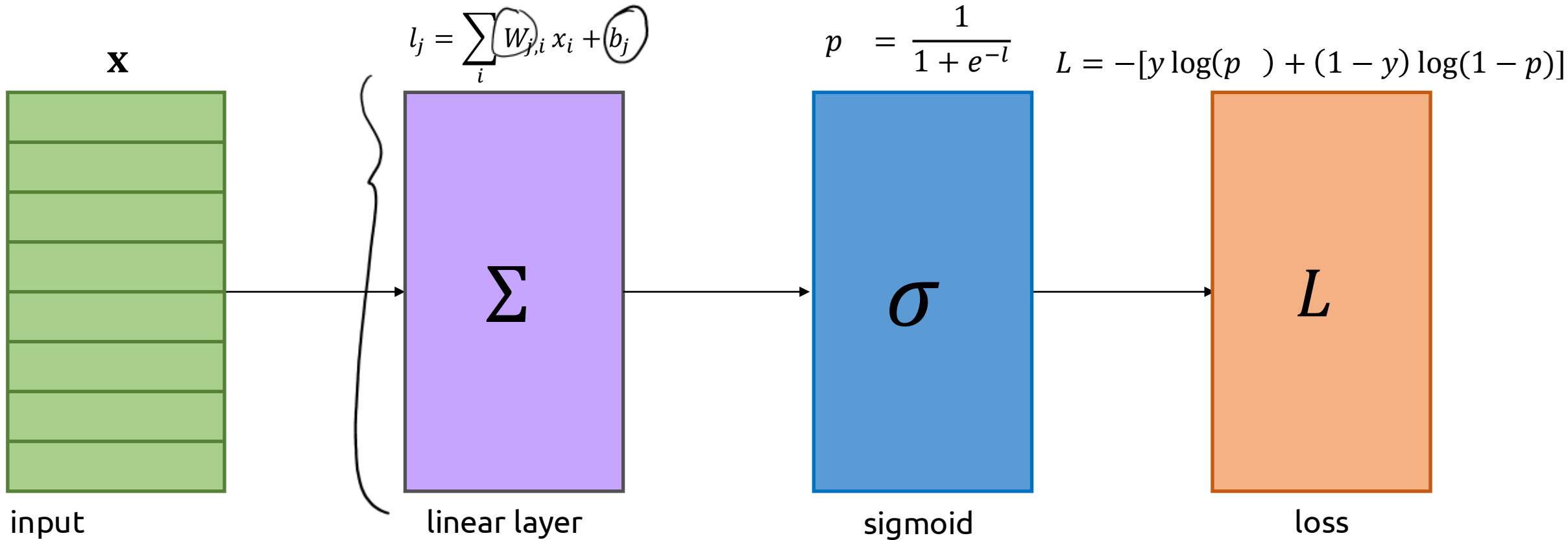
Our model after

- This is our model with the new probability layer



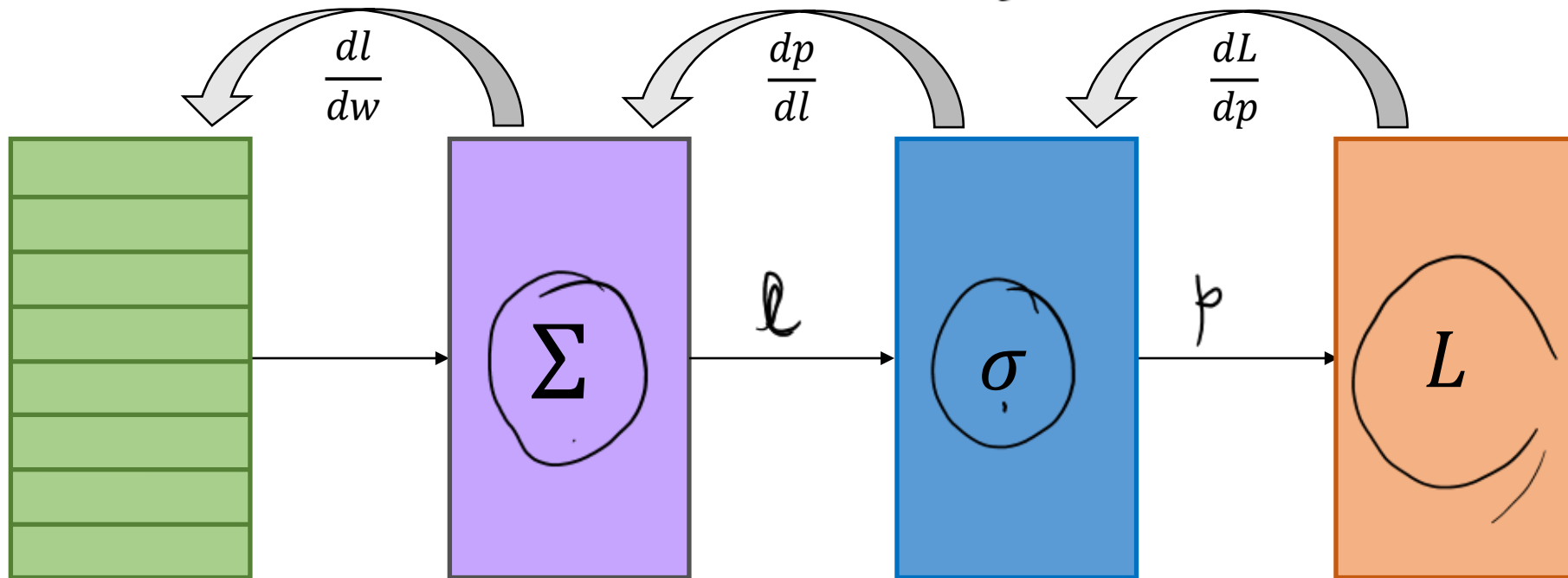
Adding Cross Entropy Loss to Our Network

Any questions?



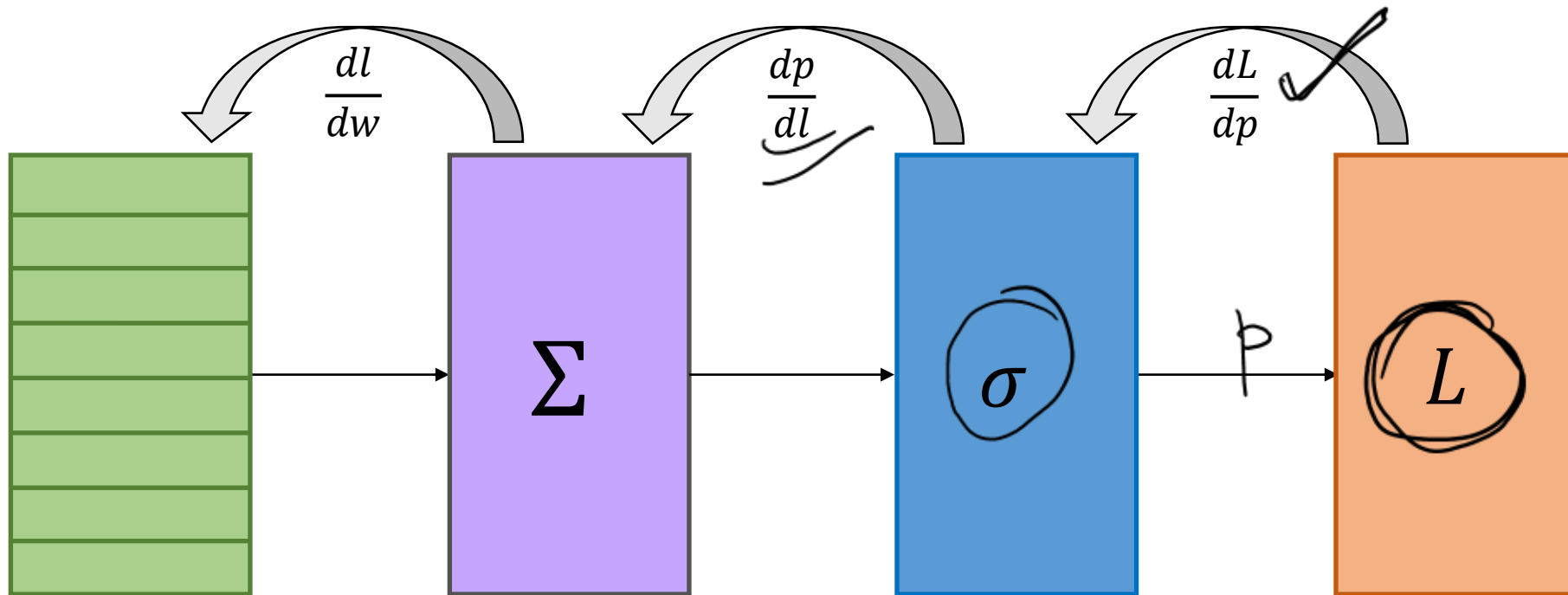
The Chain Rule in Our Network

- Here's our function: $L(p(l(w))) \Rightarrow \frac{dL}{dw} = \frac{dL}{dp} \cdot \frac{dp}{dl} \cdot \frac{dl}{dw}$



Derivative for Cross Entropy Loss Layer

$$\bullet \frac{\partial L}{\partial p} = \frac{\partial - [y \log p + (1-y) \log (1-p)]}{\partial p} = \frac{-y}{p} + \frac{1-y}{1-p}$$

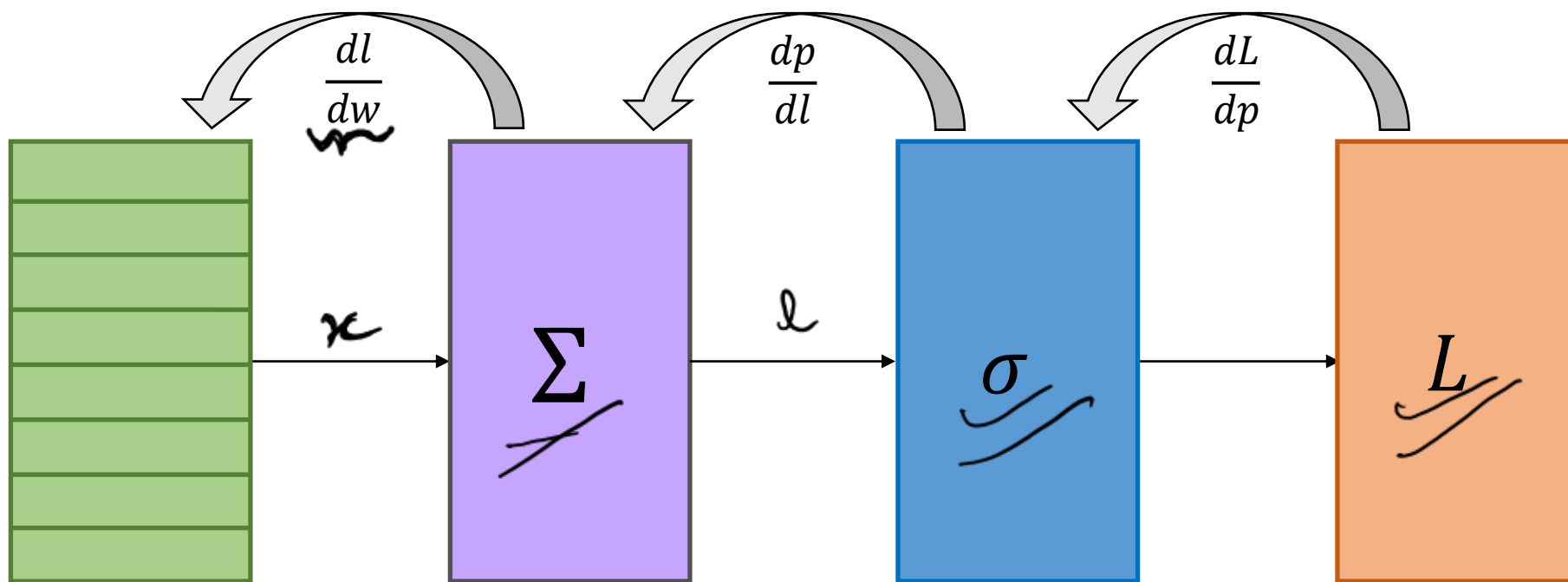


Derivative for Sigmoid Layer

$$\frac{\partial p}{\partial l} =$$

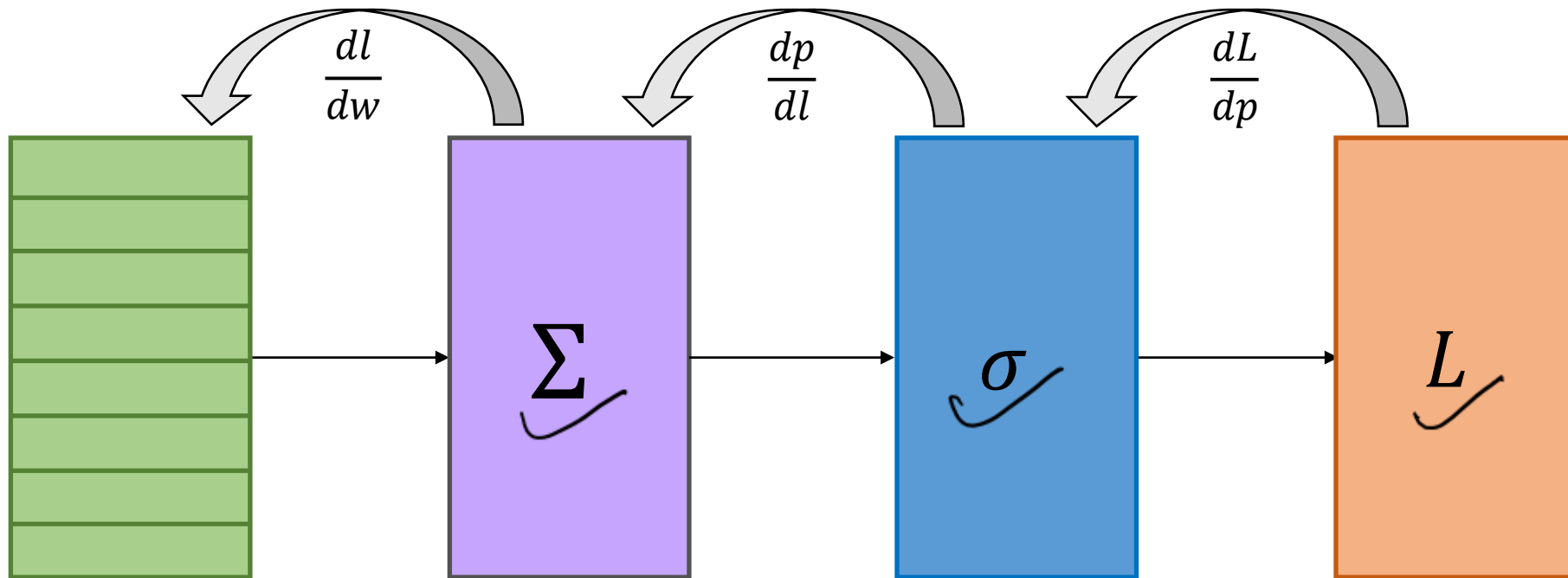
$$\begin{aligned}
 p &= a(l) = \frac{1}{1+e^{-l}} \\
 \frac{\partial p}{\partial l} &= \frac{\partial (1+e^{-l})^{-1}}{\partial l} = \frac{-1 \cdot (1+e^{-l})^{-2} \cdot (-e^{-l})}{(1+e^{-l})^2} \\
 &= \frac{e^{-l}}{(1+e^{-l})^2} = \underbrace{\frac{1}{1+e^{-l}}}_p \cdot \underbrace{\left(\frac{e^{-l}}{1+e^{-l}} \right)}_{1-p} \\
 &= p(1-p)
 \end{aligned}$$

$\frac{(1+e^{-l})-1}{(1+e^{-l})} = 1 - \underbrace{\left(\frac{1}{1+e^{-l}} \right)}_p = 1-p$



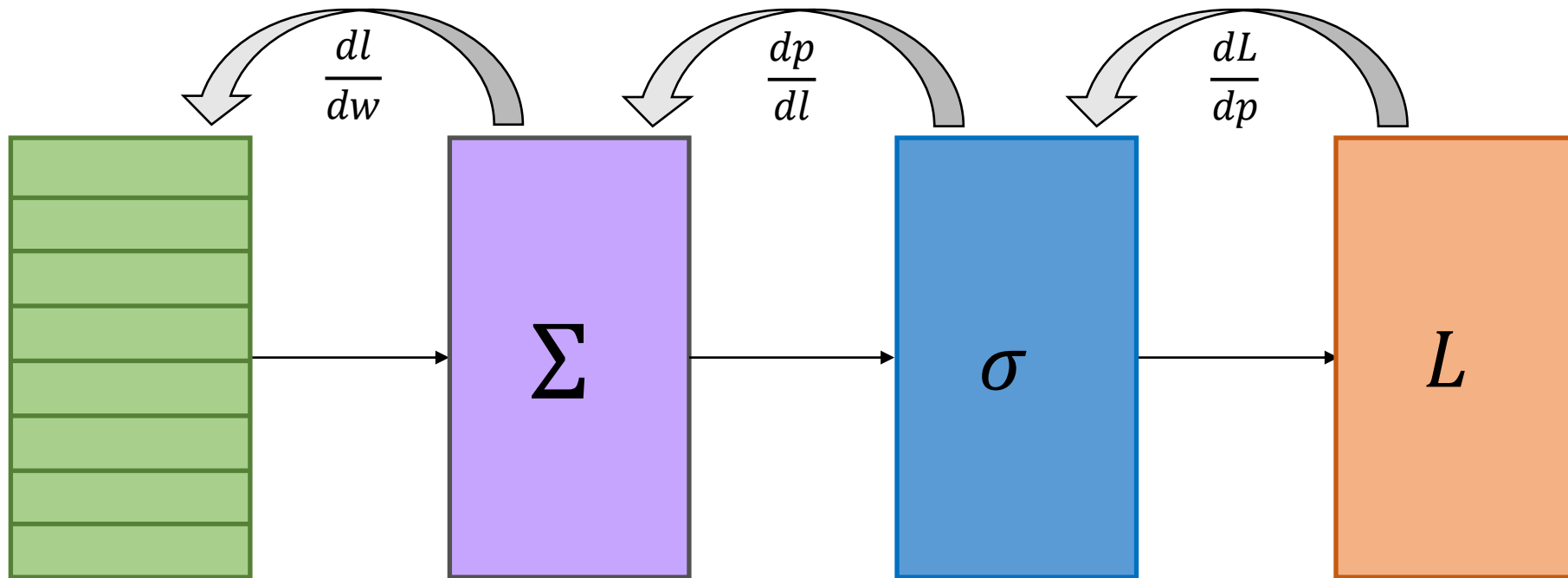
Chain Rule for Linear Layer

- $\frac{\partial l}{\partial w} = \frac{\partial wx+b}{\partial w} = x$



Chain Rule Put Together

$$\underbrace{\Delta w}_{\nearrow} = -\underbrace{\alpha}_{\nearrow} \underbrace{\frac{\partial L}{\partial w}} = -\alpha \cdot \underbrace{\frac{\partial L}{\partial p}} \cdot \underbrace{\frac{\partial p}{\partial l}} \cdot \underbrace{\frac{\partial l}{\partial w}}$$



$$\frac{\partial L}{\partial p} = \frac{-y}{p} + \frac{(1-y)}{1-p}$$

$$\left[\frac{-y}{p} + \frac{(1-y)}{(1-p)} \right] \cdot p[1-p]$$

$$\frac{\partial p}{\partial \ell} = p(1-p)$$

$$\frac{-y(1-p) + (1-y)p}{p(1-p)} \cdot \cancel{p(1-p)}$$

$$\frac{\partial \ell}{\partial w} = x$$

$$= (p - y) \cdot x$$

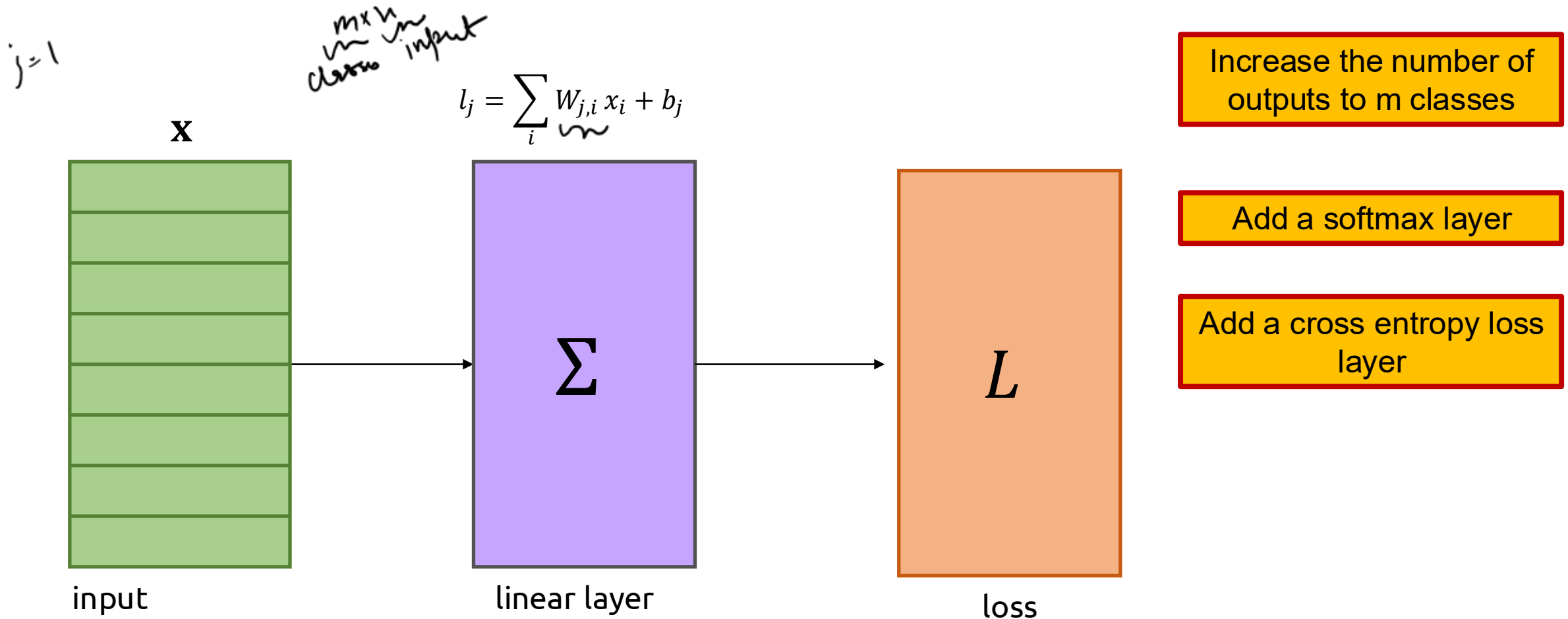
Gradient Descent: Conclusion

- Update rule: $\Delta w_{j,i} = -\alpha \cdot (p - y) \cdot x_i = \alpha \cdot (y - p) \cdot x_i$
- We use this to descend along the gradient toward the minimum loss value
- We used chain rule to propagate backwards through the entire network while doing the derivative - ***backpropagation***

Any questions?

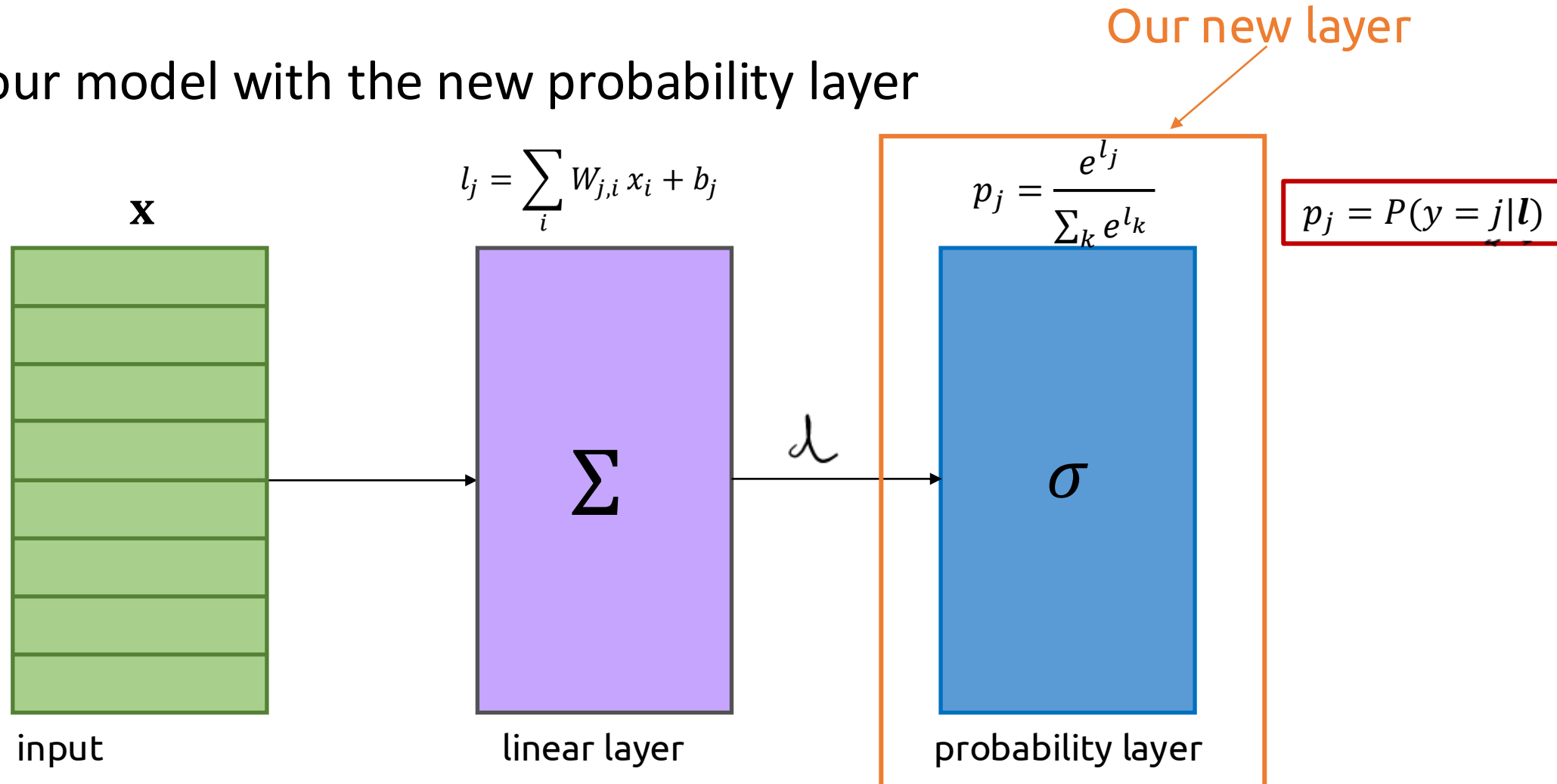


What changes do we make for scaling this multi-class classification?



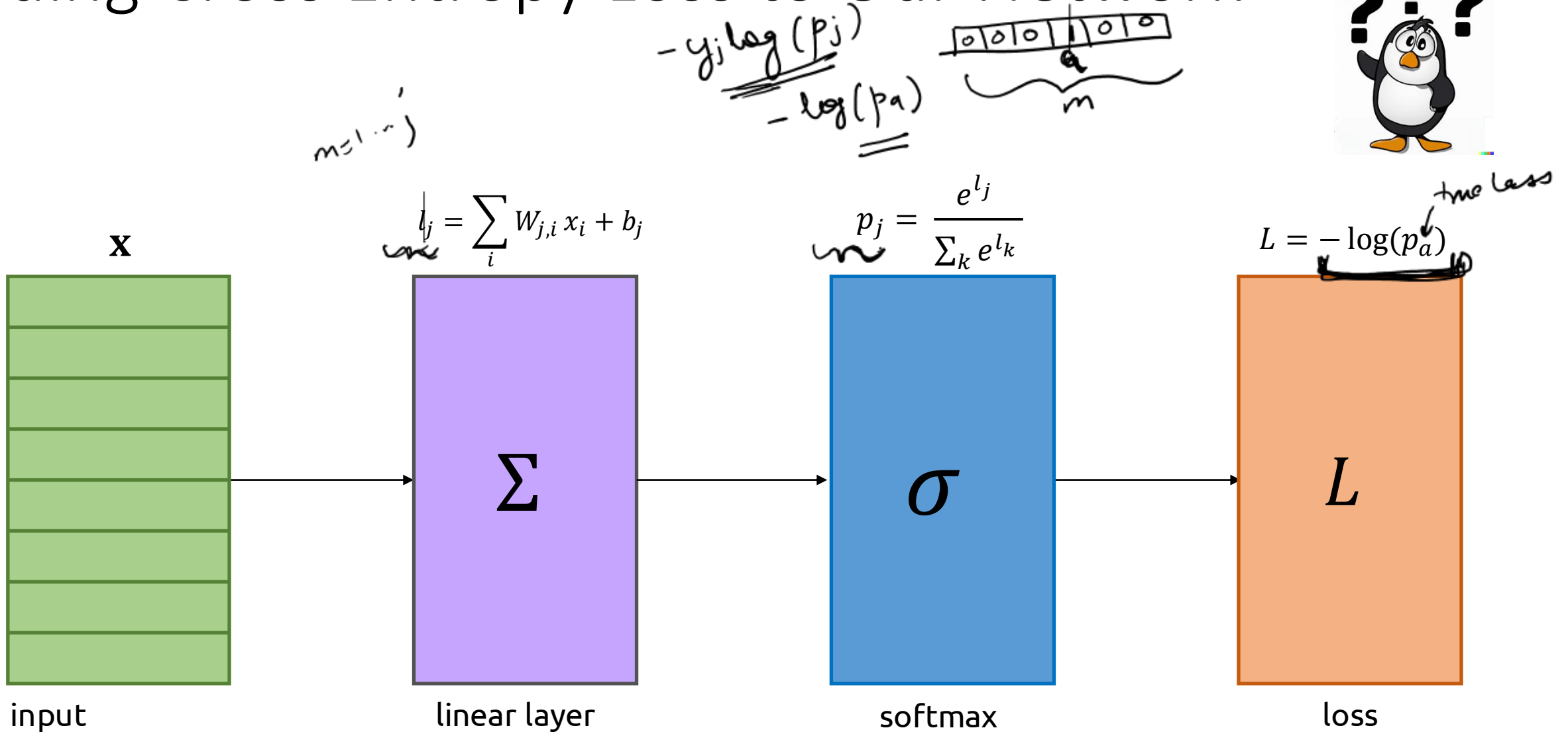
Our model after

- This is our model with the new probability layer



Adding Cross Entropy Loss to Our Network

Any questions?

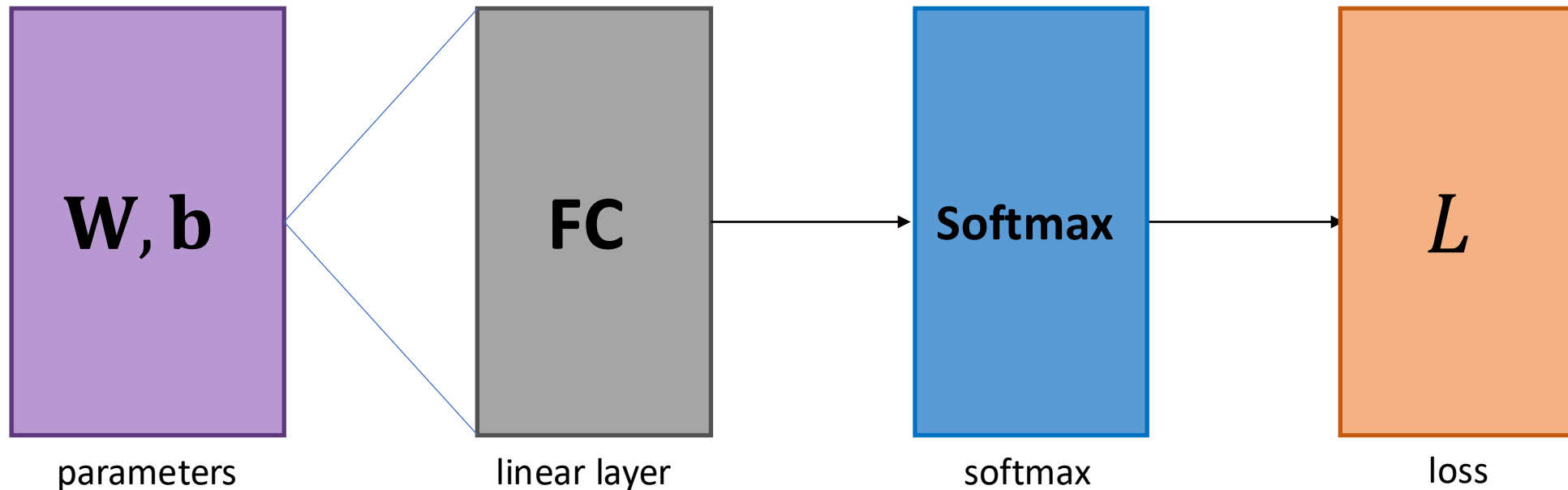


Backpropagation in action

Forward Pass

Compute the prediction or evaluate the loss for a single input x .

Goal of learning: Minimize the total loss for all x in training data with respect to model parameters W, b .



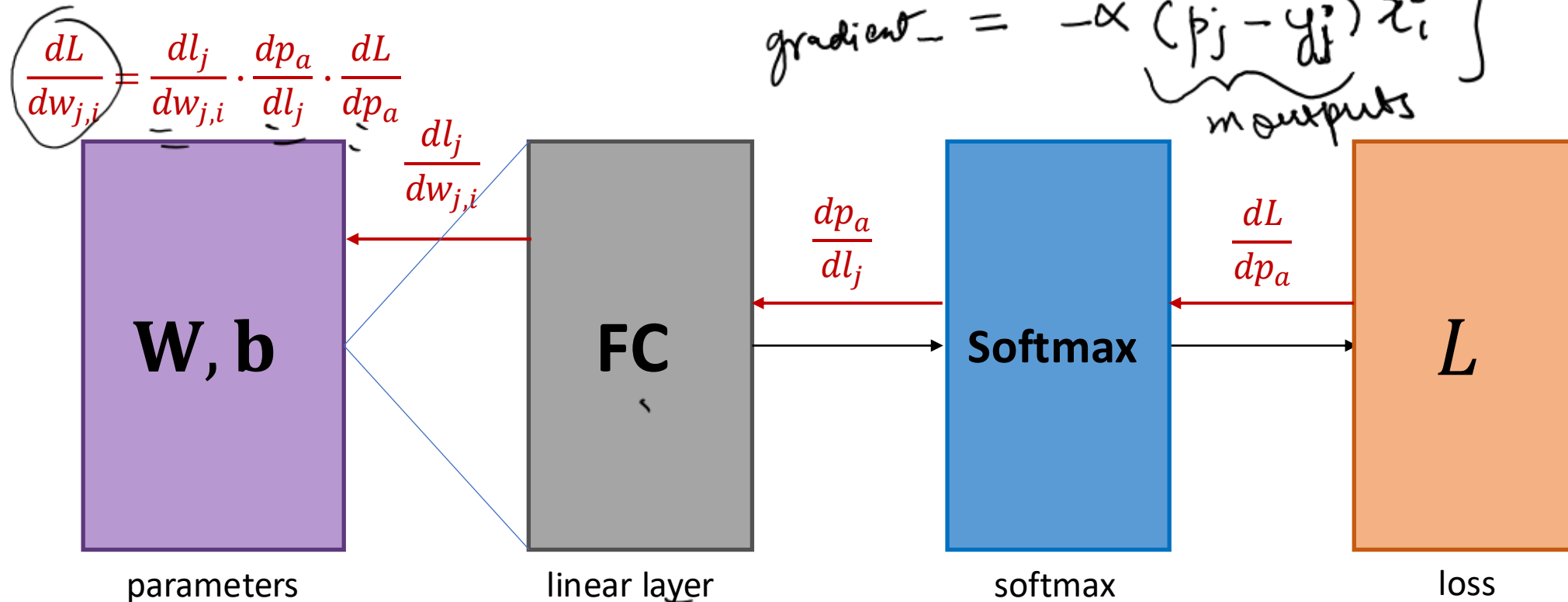
Backpropagation (Backward Pass)

binary class $\left\{ \begin{array}{l} -\alpha(p-y)x_i \\ 1 \text{ output} \end{array} \right.$

Gradient descent: $\underline{\Delta W} = -\alpha \nabla \hat{L}(W)$ and $\Delta \mathbf{b} = -\alpha \nabla \hat{L}(\mathbf{b})$

Backpropagation: Compute $\underline{\Delta W}$ and $\underline{\Delta \mathbf{b}}$ via chain rule.

gradient = $-\alpha \underbrace{(p_j - y_j^*)}_{m \text{ outputs}} x_i \left. \vphantom{p_j - y_j^*} \right\} \text{multi class}$



Putting Everything Together: Gradient Descent

delta_W is 2-D matrix of 0's in the shape of W

for each input and corresponding answer a:

```
probabilities = run_network(input)
```

Forward pass

```
for j in range(len(probabilities)):
```

```
    y_j = 1 if j == a else 0
```

```
    for i in range(len(input):
```

```
        delta_W[j][i] += alpha * (y_j - probabilities[j]) * input[i]
```

Backward pass:

Compute $\frac{\partial L}{\partial W_{ij}}$ for every W_{ij}

Over the entire dataset

```
W += delta_W
```

Putting Everything Together: Gradient Descent

delta_W is 2-D matrix of 0's in the shape of W

for each input and corresponding answer a:

```
probabilities = run_network(input)
```

Forward pass

```
for j in range(len(probabilities)):
```

```
    y_j = 1 if j == a else 0
```

```
    for i in range(len(input):
```

```
        delta_W[j][i] += alpha * (y_j - probabilities[j]) * input[i]
```

Backward pass:

Compute $\frac{\partial L}{\partial W_{ij}}$ for every W_{ij}

Over the entire dataset

```
W += delta_W
```

Gradient descent update

Gradient Descent: Limitation?

delta_W is 2-D matrix of 0's in the shape of W

for each input and corresponding answer a:

We iterate over the *entire* dataset...

probabilities = run_network(input)

for j in range(len(probabilities)):

y_j = 1 if j == a else 0

for i in range(len(input)):

delta_W[j][i] += alpha * (y_j - probabilities[j]) * input[i]

W += delta_W

...to update the weights only *once*

Stochastic Gradient Descent (SGD)

- Alternative is to train on **batches**: small subsets of the training data
- Why *stochastic*: Each batch is **randomly** sampled from the full training data
- We update the parameters after each **batch**

Stochastic Gradient Descent: Pseudocode

for each batch:

delta_W is 2-D matrix of 0's in the shape of W

for each input and corresponding answer a **in batch:**

probabilities = run_network(input)

for j in range(len(probabilities)):

y_j = 1 if j == a else 0

for i in range(len(input)):

delta_W[j][i] += alpha * (y_j - probabilities[j]) * input[i]

W += delta_W

Stochastic Gradient Descent: Pseudocode

for each batch:

delta_W is 2-D matrix of 0's in the shape of W

for each input and corresponding answer a in batch:

probabilities = run_network(input)

for j in range(len(probabilities)):

y_j = 1 if j == a else 0

for i in range(len(input)):

delta_W[j][i] += alpha * (y_j - probabilities[j]) * input[i]

W += delta_W

Now we update weights *after every batch*

Stochastic Gradient Descent (SGD)

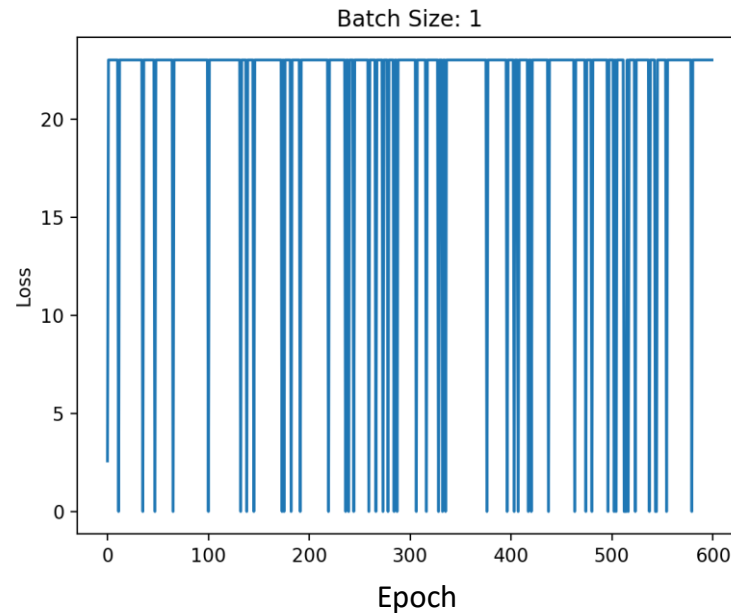
- Train on ***batches***: small subsets of the training data
- We update the parameters after each batch
- This makes the training process *stochastic* or non-deterministic: -
 - *batches are a *random* subsample of the data
 - ***do not provide the gradient that the entire dataset** as a whole would provide at once
- Formally: the gradient of a randomly-sampled batch is an unbiased estimator of the gradient over the whole dataset
 - “Unbiased”: expected value == the true gradient, but may have large variance (i.e. the gradient may ‘jitter around’ a lot)

Any questions?

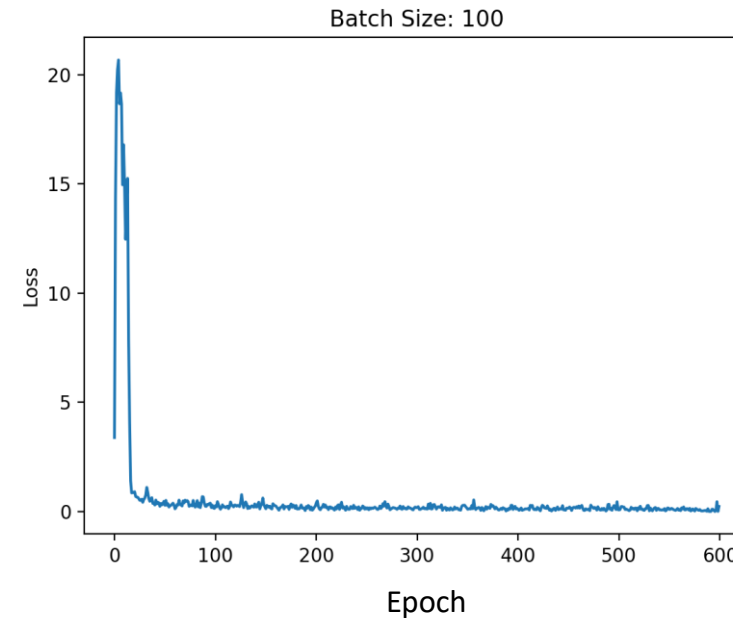


What size should the batch be?

Small batch size:
Fast, jittery updates



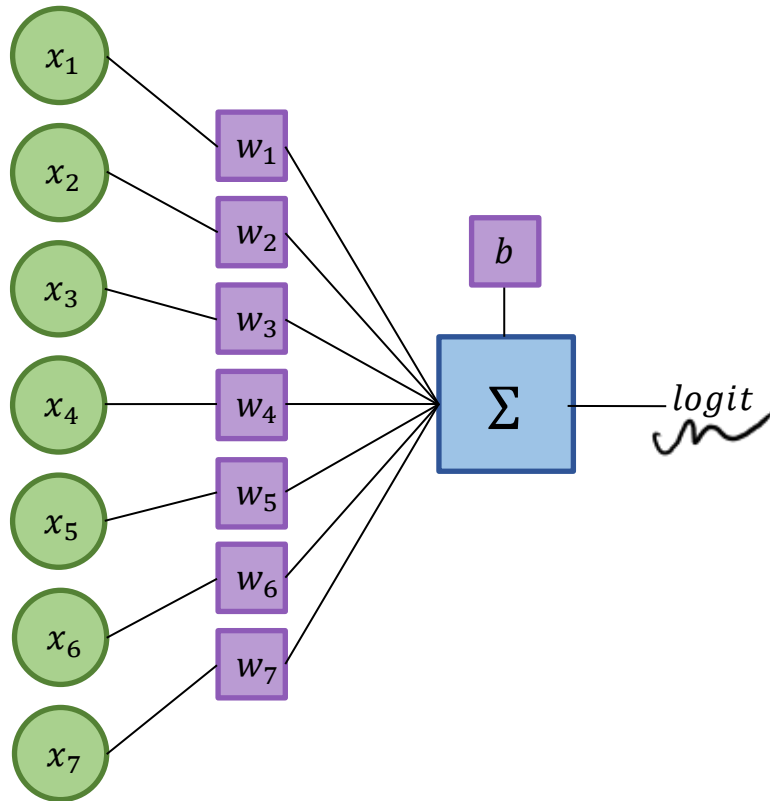
Large batch size:
Slower, stable updates



- Rule of thumb nowadays: Pick the largest batch size you can fit on your GPU!

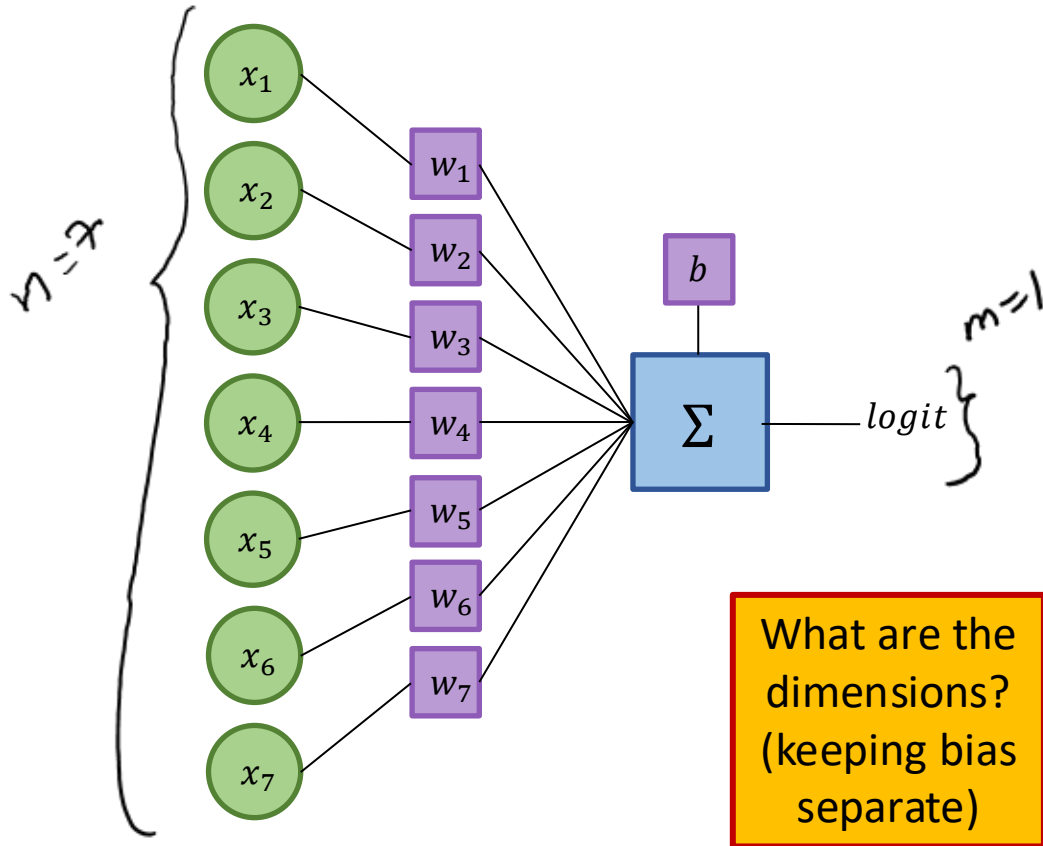
Neural networks as matrix operations

Recap: Simple Neural net (w/ linear unit)



$$\text{logit} = w_1x_1 + w_2x_2 + \cdots + w_kx_k + b$$

Matrix-multiplication Style Neural Net



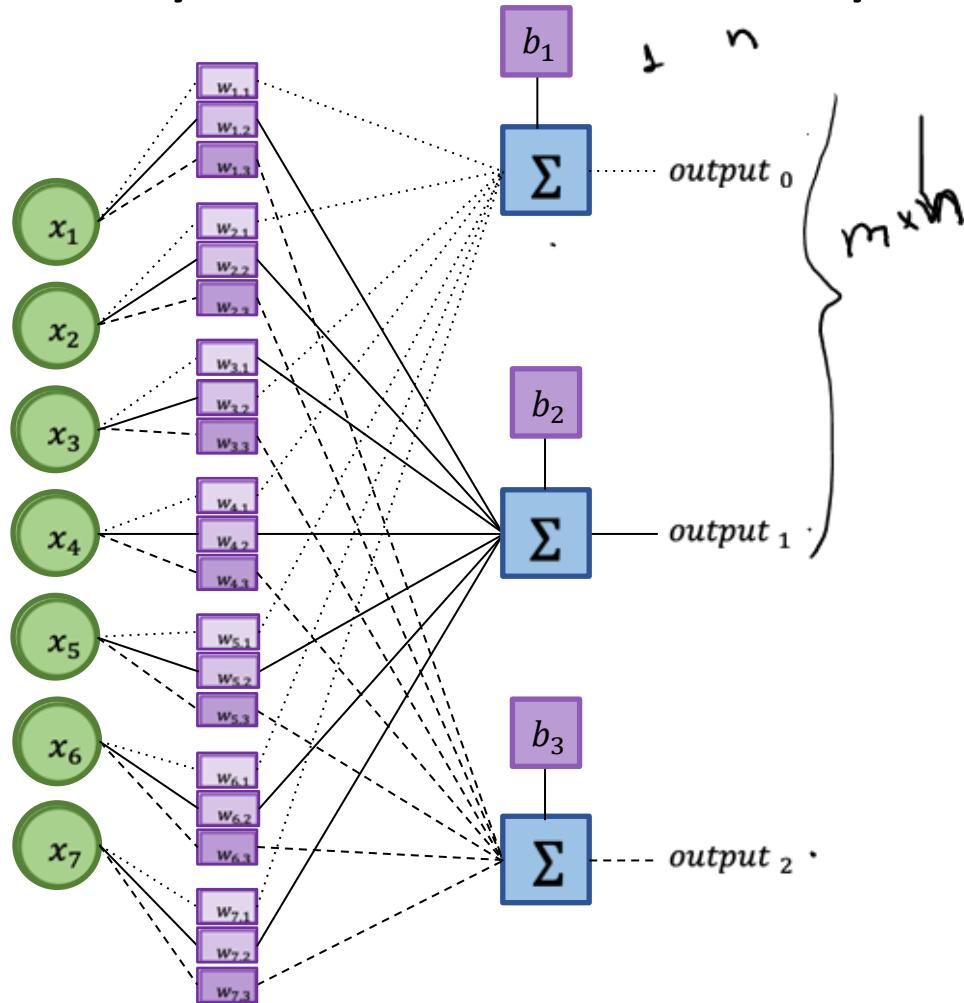
$$\text{logit} = w_1x_1 + w_2x_2 + \cdots + w_kx_k + b$$

- Really, this is just

$$\text{logit} = \overset{1 \times 7}{\underset{\text{row of } W}{W}} \overset{7 \times 1}{\underset{\text{column of } x}{x}} + \overset{1 \times 1}{\underset{\text{scalar } b}{b}}$$

- W = vector of weights (1×7 in this example)
- x = input as a vector (7×1 in this example)
- b = scalar bias (1×1)

Fully connected layer with multiple outputs



m outputs means

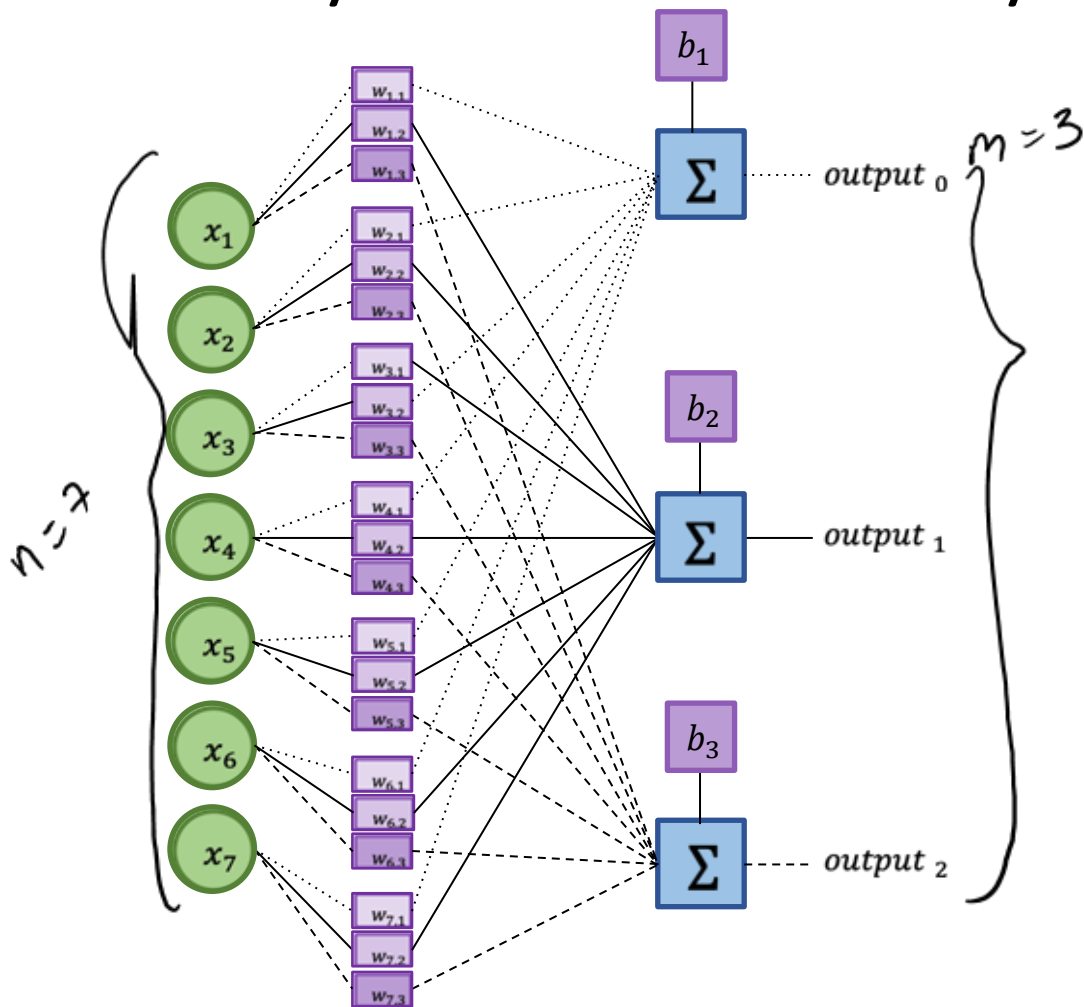
- m sets of linear functions

which means:

- m sets of weight vectors (or a weight matrix)
- m biases

What are the dimensions?

Fully connected layer with multiple outputs



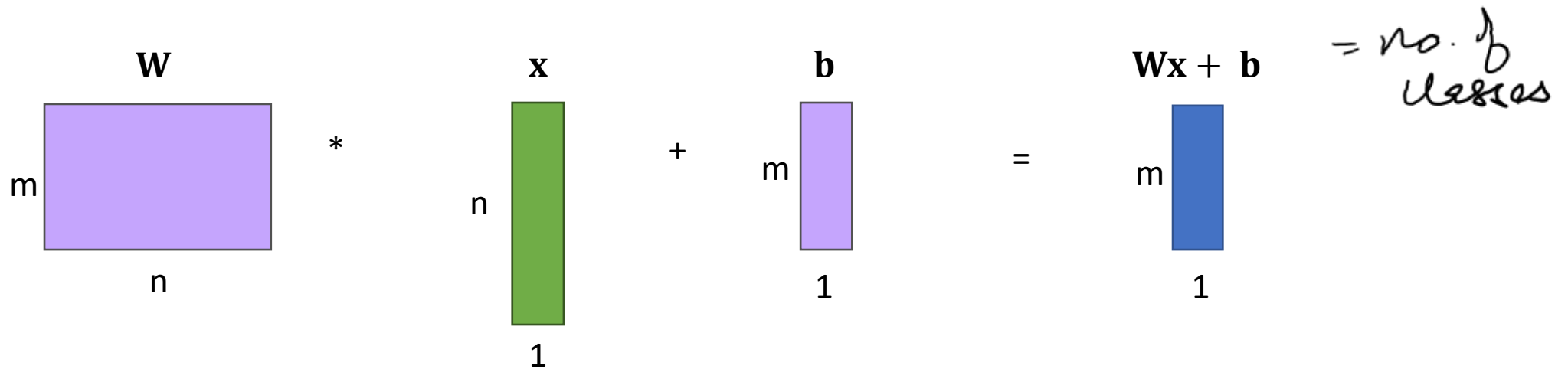
What are the dimensions?

$$logit = \overset{3 \times 7}{W} \overset{7 \times 1}{x} + \overset{3 \times 1}{b}$$

- W = matrix of weights (3×7 in this example)
- x = input as a vector (7×1 in this example)
- b = vector bias (3×1)

Fully connected layer with multiple outputs

- dimensions of $\mathbf{W} = (m, n)$
- Dimensions of $\mathbf{b} = (m, 1)$
- $\mathbf{W}\mathbf{x} + \mathbf{b}$ then is a $(m, n) * (n, 1) + (m, 1)$



Gradient Updates using Matrices

- Previously: $\Delta \mathbf{w}_{i,j} = -\alpha \cdot \frac{\partial L}{\partial w_{i,j}}$

- With Matrices: $\Delta \mathbf{W} = -\alpha \cdot \nabla_{\mathbf{w}} L$

mxn matrix of weights

mxn matrix of partial derivatives of loss w.r.t. weights

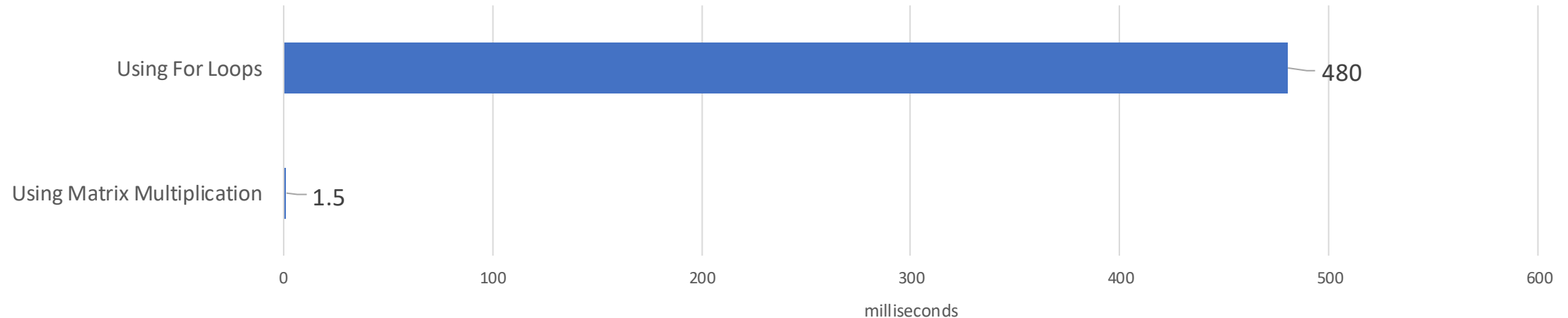
Jacobian matrix:
matrix of all first-order partial derivatives for a
vector-valued function.

i.e.
$$\begin{bmatrix} \frac{\partial L}{\partial w_{1,1}} & \frac{\partial L}{\partial w_{1,2}} & \cdots & \frac{\partial L}{\partial w_{1,n}} \\ \vdots & \vdots & & \vdots \\ \frac{\partial L}{\partial w_{m,1}} & \frac{\partial L}{\partial w_{m,2}} & \cdots & \frac{\partial L}{\partial w_{m,n}} \end{bmatrix}$$

Why is matrix formulation useful?

Existing linear algebra optimizations

- Matrix multiplication can be **way** faster than *for* loops
- Example: time required to compute dot product of $a, b \in \mathbb{R}^{1,000,000}$



From: <https://www.coursera.org/lecture/neural-networks-deep-learning/vectorization-NYnog>

- Lots of existing effort to build fast linear algebra code (e.g. NumPy)
- Leads to order of magnitude speedup!

GPUs to the rescue!



- *Graphics* Processing Units
- GPUs are really good at computing mathematical operations in parallel!
- Matrix multiplication == many **independent** multiply and add operations

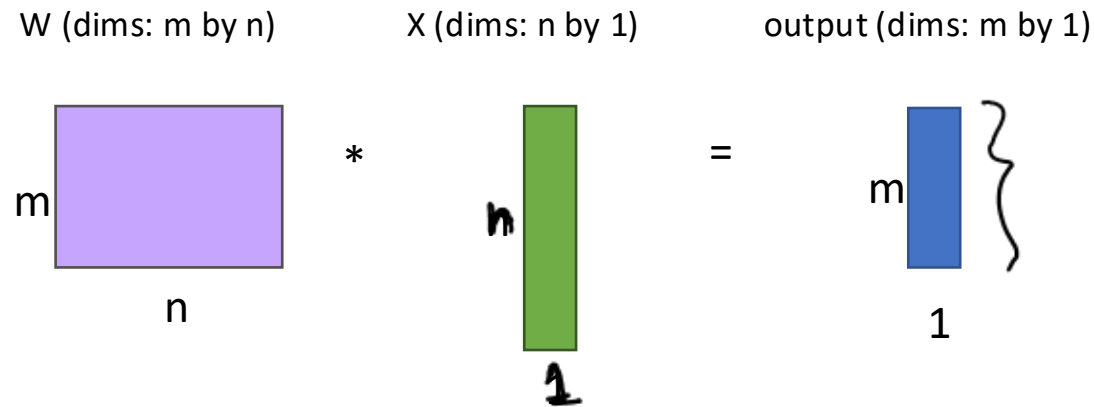
Easily parallelizable

GPUs are great for this!

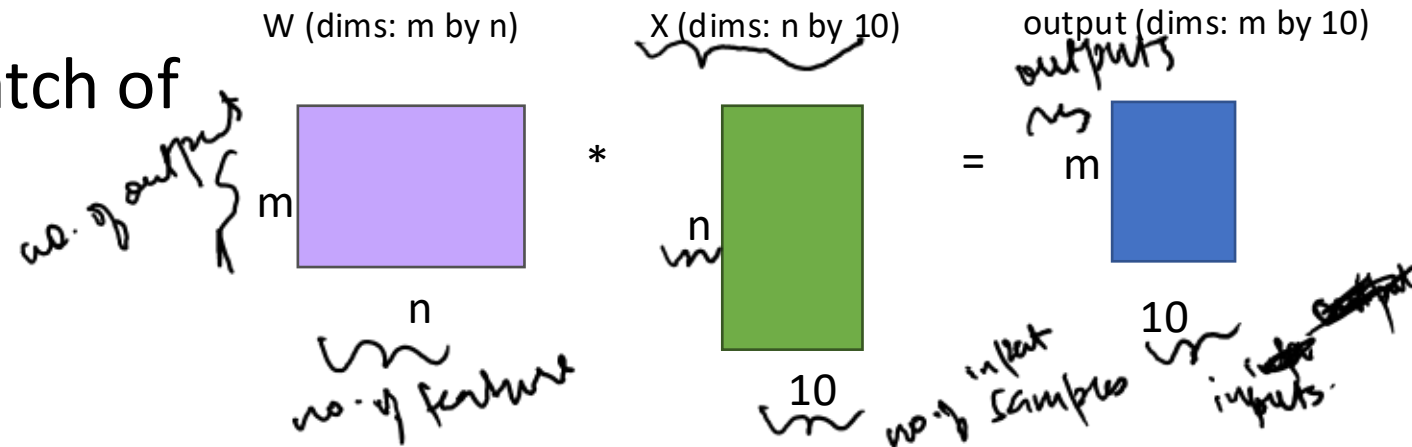
Batching and broadcasting

Computing a “batch” of outputs

- We can compute output of a single $n \times 1$ input by multiplying it by weight matrix

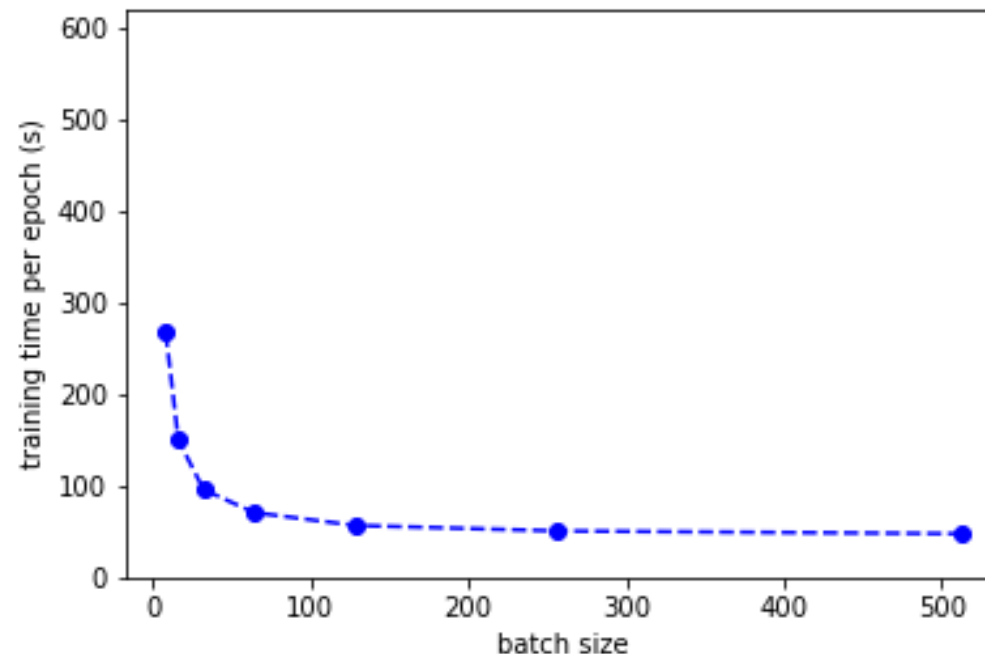


- What about a batch of 10 inputs?



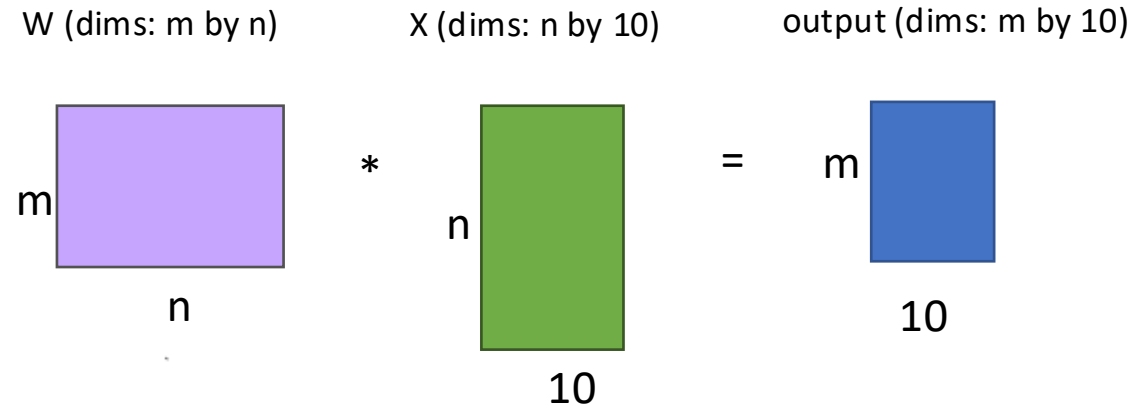
Benefit of matrices in batching

- GPU can process a whole batch in parallel!
 - In practice, we use the biggest batch size that will fit on our GPU (from last lecture)
- Example: Training duration of a CNN with GPU for different batch sizes

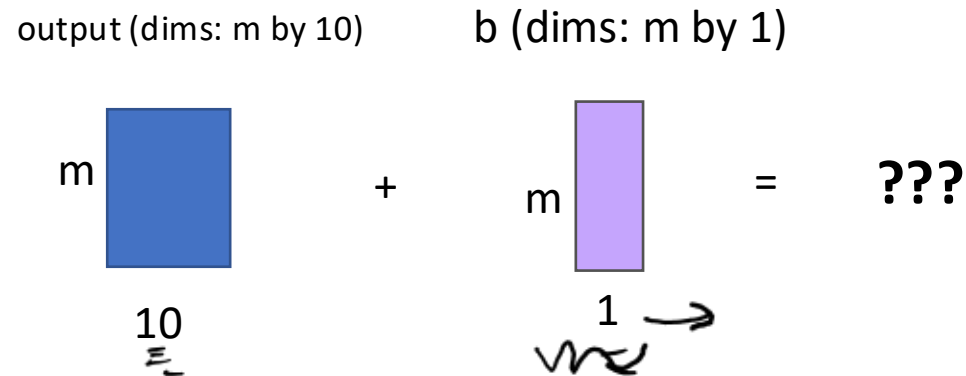


Adding a term (e.g. bias)

• $\underbrace{W} * \underbrace{x}$:



• $+ b$:



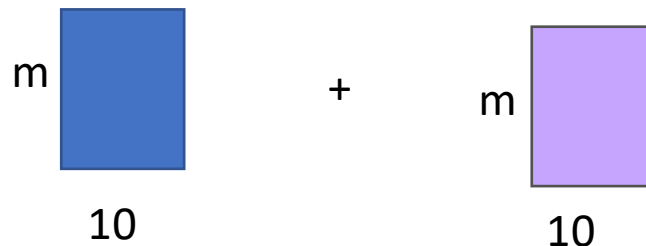
- Can't add matrices of different dimensions!
- What should we do?

Broadcasting

- Actually not a problem because of broadcasting!
- Broadcasting: implicitly replicating a tensor along some dimension to make math operations possible.
- NumPy, Tensorflow, PyTorch will all broadcast for you.

• Example: $(m, 10) + \underbrace{(1, 10)}_{\text{b}} \rightarrow (m, 10) + \underbrace{m}_{\text{output}} * (1, 10)$

• $+$ **b**: output (dims: m by 10) b (dims: m by 1)



Broadcasting

- Actually not a problem because of broadcasting!
- Broadcasting: implicitly replicating a tensor along some dimension to make math operations possible.
- NumPy, Tensorflow, PyTorch will all broadcast for you.

$$\bullet \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{bmatrix} + [100 \quad 200 \quad 300] =$$

Broadcasting

- Actually not a problem because of broadcasting!
- Broadcasting: implicitly replicating a tensor along some dimension to make math operations possible.
- NumPy, Tensorflow, PyTorch will all broadcast for you.

$$\bullet \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{bmatrix} + [100 \quad 200 \quad 300] = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{bmatrix} + \begin{bmatrix} 100 & 200 & 300 \\ 100 & 200 & 300 \\ 100 & 200 & 300 \\ 100 & 200 & 300 \end{bmatrix} =$$



Broadcasting

- Actually not a problem because of broadcasting!
- Broadcasting: implicitly replicating a tensor along some dimension to make math operations possible.
- NumPy, Tensorflow, PyTorch will all broadcast for you.

- $$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{bmatrix} + [100 \quad 200 \quad 300] = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{bmatrix} + \begin{bmatrix} 100 & 200 & 300 \\ 100 & 200 & 300 \\ 100 & 200 & 300 \\ 100 & 200 & 300 \end{bmatrix} = \begin{bmatrix} 101 & 202 & 303 \\ 104 & 205 & 306 \\ 107 & 208 & 309 \\ 110 & 211 & 312 \end{bmatrix}$$



Broadcasting in NumPy

General Broadcasting Rules:

- When operating on two arrays, NumPy compares their shapes element-wise starting with the trailing dimensions.
- Two dimensions are compatible when
 - they are equal, or
 - one of them is 1
- Dimensions with size 1 are stretched or “copied” to match the other. The size of the resulting array is the maximum size along each dimension of the input arrays.
- **Arrays do not need to have the same number of dimensions, as long as the trailing dimensions are compatible.**

Link to NumPy documentation:

<https://docs.scipy.org/doc/numpy/user/basics.broadcasting.html>

Broadcasting in NumPy

- Example:

- (m, n) array + $(n,)$ array works
- (m, n) array + $(m,)$ array doesn't work
- (m, n) array + $(m, 1)$ array works

Tensor: multi-dimensional array

- Which of the following examples work?

A: $(5, 3, 2) + (3, 2)$

B: $(5, 3, 2) + (5, 2)$

C: $(5, 3, 2) + (5, 3)$

D: $(5, 3, 2) + (5, 1, 2)$

E: $(5, 3, 2) + (1, 3, 2)$

F: $(5, 3, 2) + (5, 3, 1)$

G: $(5, 3, 2) + ()$

Broadcasting in NumPy

- Example:
 - (m, n) array + $(n,)$ array works
 - (m, n) array + $(m,)$ array doesn't work
 - (m, n) array + $(m, 1)$ array works

Any questions?



- Which of the following examples work?
 - A: $(5, 3, 2) + (3, 2) = \text{success!}$
 - B: $(5, 3, 2) + (5, 2) = \text{failure}$ 😞
 - C: $(5, 3, 2) + (5, 3) = \text{failure}$ 😞
 - D: $(5, 3, 2) + (5, 1, 2) = \text{success!}$
 - E: $(5, 3, 2) + (1, 3, 2) = \text{success!}$
 - F: $(5, 3, 2) + (5, 3, 1) = \text{success!}$
 - G: $(5, 3, 2) + () = \text{success!}$

Fully connected Neural Network for Binary Classification

Recap – Learn about classification using fully connected neural networks and some practical deep learning considerations

(1) **New task**: Predicting from medical images

(2) FCNN for classification

Homework: [Chapter 4](#) of the Dive into Deep Learning Book

(3) Backpropagation in action

(4) Neural Networks as matrix operations

(5) Batching and broadcasting

(6) **Class activity**: Coding Demo in Pytorch

Wrap up



What was the clearest point today?

What was the muddiest point today?

